Lecture 1: Introduction CS3212 Algorithms

Aya Zirikly
08/25/2026

Welcome!

Introduction to (the theory of) algorithms

- How to design algorithms
- How to analyze algorithms

Prerequisites: CSCI 1311: Discrete Structures I & CSCI 2113: Software Engineering

Instructors: Aya Zirikly

Lab Instructors & Course Graders

- Edward Bae UTA
- Han Chen LA
- Saad Mankarious GTA
- Chuhui Qiu GTA

Logistics

Class website https://azirikly.github.io/algorithms-gw26/about/

Syllabus https://azirikly.github.io/algorithms-gw26/syllabus/

- Topics
- Grading
- Policies

Labs

- Please read details on the website https://azirikly.github.io/algorithms-gw26/labs/
- Attendance is mandatory
- Git
- Python
- First lab setup

Grading

Component	Weight
Final Exam	25%
Midterm Exam	20%
Lecture Quizzes (5 total, no drop)	20%
Lab Quizzes (6 total, lowest dropped)	18%
Homework (3 total, 4% each)	12%
Participation	5%
Total	**100%**

Use of Technology

Please read all the details on the website in the Syllabus

In class

Laptops and tablets may be used for course-related purposes during lecture and lab sessions, unless the instructor explicitly states otherwise for a given activity. Phones should be put away during class — please keep them off your desk and on silent — unless needed for a specific course-related purpose.

Quizzes and lab quizzes

These are closed-book and closed-technology. No phones, laptops, tablets, or other electronic devices are allowed.

Use of Technology

Exams

Exams are closed-book and closed-technology, with one exception: you may bring one page of handwritten notes. Typed or printed materials — including printouts of the slides — are not permitted and will result in a grade of zero on the exam. Because only handwritten notes are allowed, you’re encouraged to take notes by hand throughout the semester to prepare.

Module 1

Introduction, Efficiency, and Asymptotic Foundations

Goal

Transition from writing code that works to mathematically analyzing how algorithms scale.

Core philosophy

Programming is language-dependent; algorithm analysis is mathematical and invariant to hardware or runtime environments.

Roadmap

- ❏ What is an algorithm? (Specification vs. Implementation)
- ❏ Why empirical clock-time benchmarking fails
- ❏ The RAM (Random Access Machine) model of computation
- ❏ Asymptotic notation definitions: Big-O, Big-Ω, and Big-Θ
- ❏ Step-by-step mathematical proofs finding constants c and n_0
- ❏ The asymptotic growth rate hierarchy

What is an Algorithm?

- Unambiguous, step-by-step computational procedure that takes a set of values as **input** and produces a set of values as **output** in a finite number of steps.

- **The Algorithmic Lifecycle:**

Real-World Problem → Formal Model → Algorithm Design → Asymptotic Analysis → Code

Examples

TikTok Video Rendering & Feed Sorting

- **The Real-World Problem:** How do you select the exact next 5-second video clip to show a user to maximize their watch time?
- **The Algorithmic Approach:** An **Exploration-Exploitation Loop** (Multi-Armed Bandit problem). The algorithm feeds you 80% videos it ***knows*** you like (Exploitation) and 20% wildcards in new topics (Exploration) to test if your interests have shifted, tracking watch time down to the millisecond.

Measuring Algorithm Performance

Correctness: Does the algorithm yield the valid result for ***every*** allowed input instance?

Efficiency: How do time and memory requirements scale as input size n $\rightarrow \infty$?

Correctness

- **Allowed Input Instance:** Any data that fits the problem's input requirements.
  - *Example:* If a sorting algorithm specifies its input as "an array of numbers," then an empty array `[]`, an array with one element `[42]`, an array with duplicates `[3, 1, 3]`, and an array with negative numbers `[-5, 0, 2]` are all legal input instances.
- **Valid Result:** An output that satisfies the problem's goal. For sorting, the valid result is the exact same set of numbers arranged in monotonic order.
- **For EVERY Legal Input Instance:** It is not enough for an algorithm to work on typical or "average" test cases. It must handle **edge cases** and worst-case scenarios without crashing, looping infinitely, or outputting incorrect data.

What is the "Correct" Answer?

In formal theoretical computer science, the answer must be **YES**, which is proven using two components:

1. **Partial Correctness:** *If* the algorithm terminates, the result it outputs is guaranteed to be correct.
2. **Termination:** The algorithm is guaranteed to stop executing after a finite number of steps (it will never enter an infinite loop).

When an algorithm satisfies both (Terminates + Valid Output), it achieves **Total Correctness**

When Would the Answer Be "No"?

When Would the Answer Be "No"?

A Buggy/Incorrect Algorithm: If an algorithm fails for even *one* single legal/allowed input (for example, it can sort positive numbers, but corrupts data when given negative numbers).

When Would the Answer Be "No"?

A Buggy/Incorrect Algorithm: If an algorithm fails for even *one* single legal/allowed input (for example, it can sort positive numbers, but corrupts data when given negative numbers).

Approximation & Randomized Algorithms: Higher-level algorithms (like those used in machine learning, NP-hard problems, or cryptography) intentionally trade absolute correctness for speed. These accept answers like *"No, but it gets within 99% of the right answer fast"* (Approximation Algorithms) or *"No, but it gives the right answer 99.999% of the time"* (Monte Carlo Algorithms).

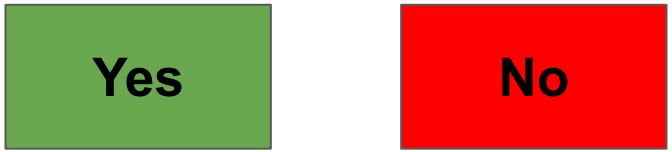
Google Maps doesn't guarantee the 100% single best route because finding it triggers a **combinatorial explosion**—checking every possible road combination across millions of streets would take hours. Instead, it uses a quick approximation that gives you a 98% optimal route in under a second.

Find Median

```
def find_middle(arr):
    mid_index = len(arr) // 2
    return arr[mid_index]
```

Integer division

Correct?



```python
def find_middle(arr):
    mid_index = len(arr) // 2
    return arr[mid_index]
```

- arr = [10, 20, 30] → Length is 3. 3 // 2 = 1. Returns arr[1] which is 20. **(Correct)**
- arr = [5, 10, 15, 20, 25] → Length is 5. 5 // 2 = 2. Returns arr[2] which is 15. **(Correct)**

Edge cases

Edge Case 1: Arrays with an EVEN number of elements

- **Input:** [10, 20, 30, 40] (Length = 4)
- **What the code does:** 4 // 2 = 2. Returns arr[2] which is 30.
- **Why it's wrong:** For an even number of sorted elements, the mathematical median is the *average of the two middle values*: (20 + 30) / 2 = 25. Returning 30 is mathematically incorrect.

Edge Case 2: Unsorted arrays (if "sorted" wasn't enforced by precondition)

- **Input:** `[50, 10, 90]`
- **What the code does:** Returns `arr[1]` which is `10`.
- **Why it's wrong:** The median of these numbers is `50`. If the input isn't strictly pre-sorted, the positional middle is meaningless.

Edge Case 3: Empty Array (Boundary Edge Case)

- **Input:** `[]` (Length = 0)
- **What the code does:** `0 // 2 = 0`. Tries to access `arr[0]`, throwing an `IndexError`.
- **Why it's wrong:** The algorithm crashes entirely instead of handling the boundary gracefully.

```python
def find_middle_correct(arr):
    if not arr:
        return None  # Handle empty array

    n = len(arr)
    mid = n // 2

    # Even length: average of the two middle elements
    if n % 2 == 0:
        return (arr[mid - 1] + arr[mid]) / 2.0
    # Odd length: exact middle element
    else:
        return arr[mid]
```

What Does Algorithmic Efficiency Mean?

Beyond Correctness: An algorithm that produces valid results is useless if it takes 100 years or requires more RAM than exists on Earth.

The Two Resources of Efficiency:

1. **Time Complexity:** How does the *execution time* scale as input size n increases?
2. **Space Complexity:** How does the *memory footprint* scale as input size n increases?

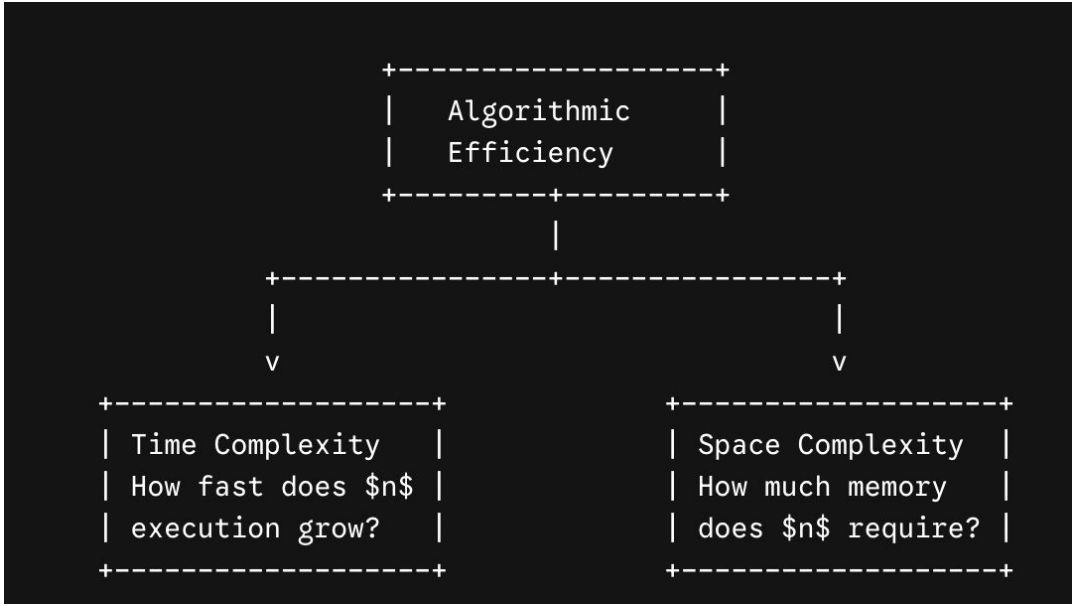
What Does Algorithmic Efficiency Mean?

Beyond Correctness: An algorithm that produces valid results is useless if it takes 100 years.

The Two Resources of Efficiency:

1. **Time Complexity:** How does the *execution time* scale as input size n increases?
2. **Space Complexity:** How does the *memory footprint* scale as input size n increases?

What Does Algorithmic Efficiency Mean?



We do not measure raw seconds; we measure **growth rate** relative to input size n

Why Efficiency Trumps Hardware Speed

The Scalability Trap: Can't we just buy a $10,000 supercomputer to solve slow algorithms?
The Math: Consider sorting n = 100,000,000 items (100 million)

Algorithm Strategy	Complexity Class	Operations Required	Execution Time (10^9 ops/sec)
Bubble Sort	O(n^2)	10^16	~115 days 10,000,000 seconds
Merge Sort	O(n log2 n)	2.7x10^9	~2.7 seconds

The Problem: Find Duplicate Elements in a List

```python
# ========================================================
# APPROACH 1: Nested Loops (Brute Force)
# Time Complexity:  O(n^2) - Quadratic
# Space Complexity: O(1)   - Constant extra space
# ========================================================
def has_duplicates_slow(arr):
    n = len(arr)
    # Compare every element with every other element after it
    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] == arr[j]:
                return True  # Found duplicate
    return False
```

```python
# ========================================================
# APPROACH 2: Hash Set Lookups
# Time Complexity:  O(n) - Linear
# Space Complexity: O(n) - Linear extra space for the set
# ========================================================
def has_duplicates_fast(arr):
    seen = set()
    # Single scan through the array
    for item in arr:
        if item in seen:  # O(1) average-case hash table lookup
            return True
        seen.add(item)  # O(1) average-case hash table insertion
    return False
```

Key takeaways

1. **Why Approach 1 is O(n^2):**
   The outer loop runs n times. For each iteration, the inner loop scans the remaining elements. The total comparisons equal:
   $(n - 1) + (n - 2) + ... + 1 = n(n - 1) / 2 = \Theta(n^2)$
2. **Why Approach 2 is O(n):**
   The loop visits each element exactly once (n iterations). Searching inside a Python set() uses a **hash table**, making item in seen an O(1) constant-time operation on average.
3. **The Space-Time Tradeoff:**
   Approach 2 achieves O(n) time speed by trading memory O(n) extra space to store the set. This is a classic engineering trade-off: **spend space to save time**.

Problem vs. Algorithm

The Difference:

A **Problem** defines *WHAT* must be accomplished (inputs and required outputs).

An **Algorithm** is the specific step-by-step procedure specifying *HOW* to solve it.

Problem vs. Algorithm

Problem Specification: Integer Sorting

- **Input:** A sequence of n real numbers [a1, a2, ..., an].
- **Output:** A permutation [a'1, a'2, ..., a'n] such that a'1 <= a'2 <= ... <= a'n.

Different Algorithms for the Same Problem:

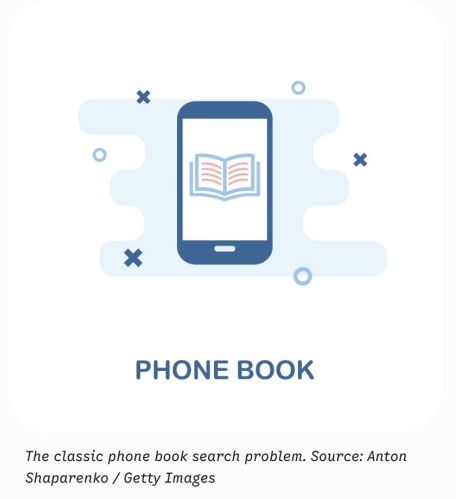
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort

Goal: Determine which algorithm is superior as input size n scales to millions of items without implementing all of them first.

Problem vs. Algorithm

- **The Problem (WHAT):** Find a person named "Zack Smith" in a physical 1,000-page phone book.
- **The Goal:** Output Zack's phone number or confirm he isn't listed.

Feature	Strategy A: Page-by-Page	Strategy B: The Half-Divide (Binary Search)
How It Works	Start at page 1 ("Adams"), check every single page one by one until you reach "Smith".	Open to the middle (page 500 - "Miller"). "Smith" comes after "M", so throw away pages 1–500 and repeat on the right half.
Worst Case	Must flip through **1,000 pages**.	Reaches the page in at most **10 cuts** ($2^{10} = 1,024$).
Efficiency	O(n) — Linear Time	O(log n) — Logarithmic Time
Result	100% Correct	100% Correct



The classic phone book search problem. Source: Anton Shaparenko / Getty Images

The Pitfalls of Empirical Benchmarking

Why don't we just measure execution time using system clocks or a stopwatch?

1. **Hardware Heterogeneity:** A CPU with a 4.5 GHz clock speed runs code faster than a 2.0 GHz processor.
2. **System Noise:** Operating system background tasks, context switching, and memory caching introduce random noise.
3. **Language & Environment Overhead:** Python interpreted bytecode vs. C compiled machine code creates huge static performance gaps.
4. **Input Size Distortion:** An O(n^2) algorithm might run in 0.001 seconds for n = 10, masking catastrophic delays when n = 1,000,000.

The Theoretical Solution — The RAM Model

To analyze algorithms independently of hardware, we use the **Random Access Machine (RAM) Model**:

- **Primitive Operations:** Instructions take 1 unit of time (O(1) constant cost):
  - Arithmetic operations (+, -, *, /)
  - Data assignments (x = y)
  - Array indexing (arr[i])
  - Pointer dereferencing
  - Conditional branching comparison (if x > y)
- **Memory Access:** Reading or writing any memory address takes O(1) time.
- **Total Running Time:** Sum of primitive operations executed as a function of input size n.

Introduction to Asymptotic Analysis

Asymptotic analysis studies how the running time T(n) grows in the **limit** as input size n → infinity.

Key Principles:

1. **Ignore Low-Order Terms:** As n grows, n^2 completely dominates 100n + 5000

   Example: T(n) = 0.0001n^3 + 100n^2 + 5000
   As n → infinity, the n^3 term dominates, making this a cubic algorithm.

2. **Ignore Constant Coefficients:** We drop scalar multipliers because hardware differences scale performance by constants anyway (5n^2 → n^2).

Asymptotic Upper Bound — Big-O

Asymptotic Upper Bound — Big-O

Big-O (O): The **Ceiling** (Worst-Case / Upper Bound).

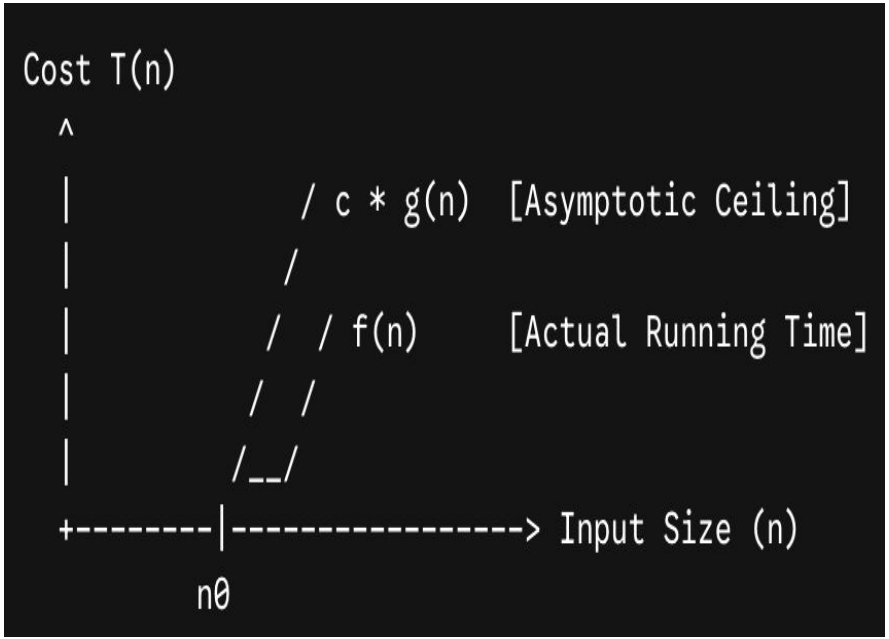
"The runtime will not be worse than this."

Asymptotic Upper Bound — Big-O

- **Formal Definition:** Let f(n) and g(n) be functions mapping non-negative integers to real numbers. We write f(n) = O(g(n)) if there exist positive constants c > 0 and n0 >= 0 such that:

  0 <= f(n) <= c * g(n) for all n >= n0

- **Intuition:** g(n) serves as an **asymptotic upper bound** (ceiling) on f(n). f(n) grows no faster than g(n).

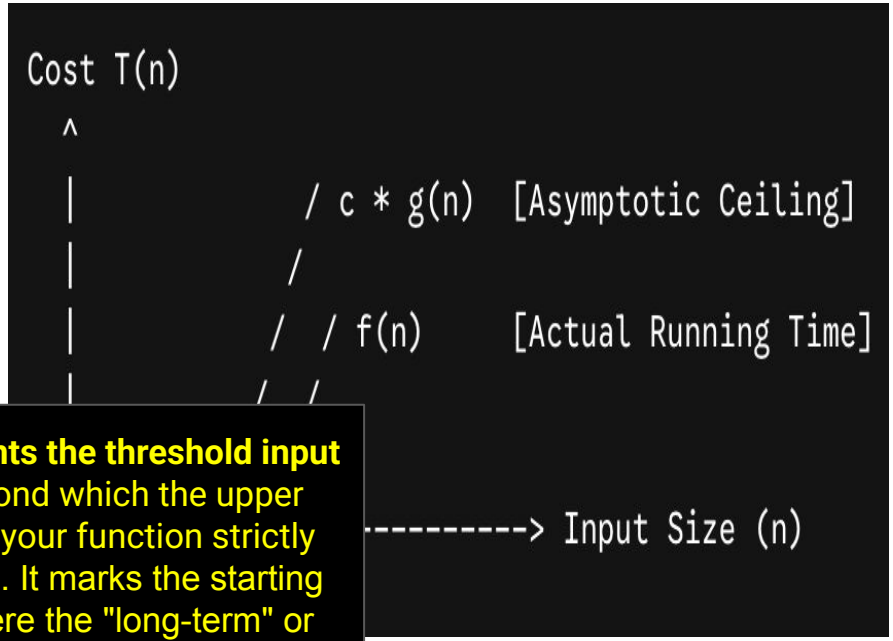


Asymptotic Upper Bound — Big-O

- **Formal Definition:** Let f(n) and g(n) be functions mapping non-negative integers to real numbers. We write f(n) = O(g(n)) if there exist positive constants c > 0 and n0 >= 0 such that:

  0 <= f(n) <= c * g(n) for all n >= n0

- **Intuition:** g(n) serves as an **asyn**
  upper bound (ceiling) on f(n). f(n
  no faster than g(n).



n_0 represents the threshold input size beyond which the upper bound of your function strictly holds true. It marks the starting point where the "long-term" or asymptotic behavior of the algorithm begins, allowing you to safely ignore smaller inputs where minor terms might fluctuate

Big-O Step-by-Step Proof Example 1 (Linear)

- **Claim:** Prove that f(n) = 4n + 7 = O(n).
- **Goal:** Find explicit constants c > 0 and n0 >= 0 satisfying 4n + 7 <= c * n for all n >= n0.

Step-by-Step Proof:

1. Assume n >= 1. Observe that 7 <= 7n.
2. Substitute 7n in place of 7:
   4n + 7 <= 4n + 7n = 11n
3. Compare directly to definition: 4n + 7 <= 11n holds for c = 11 and n0 = 1.
4. Both constants are positive real numbers.

Conclusion: f(n) = 4n + 7 = O(n) [Q.E.D.]

Big-O Step-by-Step Proof Example 2 (Quadratic)

- **Claim:** Prove that f(n) = 3n^2 + 8n + 5 = O(n^2).
- **Goal:** Find c, n0 such that 3n^2 + 8n + 5 <= c * n^2 for all n >= n0.

Step-by-Step Proof:

1. Assume n >= 1. Establish bounds for low-order terms relative to n^2:
    - 8n <= 8n^2
    - 5 <= 5n^2
2. Sum the inequalities:
   3n^2 + 8n + 5 <= 3n^2 + 8n^2 + 5n^2 = 16n^2
3. Set c = 16 and n0 = 1.
4. Check condition: 3(1)^2 + 8(1) + 5 = 16 <= 16(1)^2. Holds.

Conclusion: f(n) = 3n^2 + 8n + 5 = O(n^2) [Q.E.D.]